INFSCI 1540 - Data Engineering
Climate Data Warehouse Project
Final Report

Boyi Sun Murphy Zhao

April 20, 2026

1. Project Overview

This project builds a climate data warehouse based on NOAA Climate Data Online data. The whole project can be founded in It uses Docker, Kafka, ksqlDB, MySQL, and Python to support data ingestion, cleaning, transformation, and storage. The system organizes climate normal data in a STAR schema with fact and dimension tables, and also creates pre-aggregated monthly summary tables for efficient analysis. By integrating multiple weather stations into the pipeline, the project supports comparative climate analysis across locations and time periods. All project materials are available in the project GitHub repository: https://github.com/Boyiiiisun/INFSCI1540-Project-Climate.

2. Data Description

This project maintains climate normal data derived from the NOAA Climate Data Online source. The data source is available at https://www.ncei.noaa.gov/cdo-web/, and the implementation in the current repository is based on the `NORMAL_DLY` sample file, which stores daily normal measurements for multiple weather stations. The data pipeline currently preserves four stations in the repository sample: *PETERSBURG 2 N ND US*, *PITTSBURGH INTL AP PA US*, *SEATTLE URBAN SITE WA US*, and *SAN FRANCISCO OCEANSD CA US*. Each record is associated with one station and one calendar date.

The source file contains the following attributes: station identifier, station name, elevation, latitude, longitude, date, daily minimum temperature normal, daily maximum temperature normal, and month-to-date precipitation normal. The system preserves station metadata together with daily climate observations so that the data warehouse can support time-based analysis, station-based comparison, and pre-aggregated monthly summaries.

The grain of the main fact data in the warehouse is one climate normal observation for one station on one date. This design allows the system to support both daily-level analysis and month-level summaries derived from the daily observations.

3. Docker Containers Used in the System and Corresponding `docker-compose.yml`

The system is deployed through Docker Compose. The repository uses containerized infrastructure to isolate operational data storage, warehouse storage, streaming middleware, and query interfaces. The main containers are summarized below.

3.1 Container Roles

- **web-server**: a lightweight Apache/PHP container used as a simple web server.
- **mysql-odb-server**: the MySQL instance that stores the operational database (ODB) raw table.
- **mysql-dw-server**: the MySQL instance that stores the dimensional warehouse tables and summary tables.
- **phpmyadmin-odb**: a browser-based administration interface for the ODB MySQL instance.
- **phpmyadmin-dw**: a browser-based administration interface for the DW MySQL instance.
- **zookeeper**: the coordination service required by Kafka in this environment.
- **broker**: the Kafka broker that receives raw JSON messages from the producer and exposes them to downstream consumers and ksqlDB.
- **ksqldb-server**: the ksqlDB server used for stream definition, cleaning, transformation, and monthly aggregation.
- **ksqldb-cli**: the command-line client used to execute ksql statements interactively.

This container layout separates storage and streaming concerns. The ODB is used as a staging layer for cleaned operational records, while the DW stores the star schema and summary tables. Kafka and ksqlDB provide the streaming and transformation layer between ingestion and warehouse analytics.

After running `docker-compose up -d` in the terminal, the Docker environment starts all required services for the project. The screenshot below shows the running containers, confirming that the ODB, DW, Kafka, ksqlDB, phpMyAdmin, and web-server components were successfully launched before the data preparation and warehouse loading steps.

```
PS C:\Users\孙博弈\Desktop\INFSCI 1540\Project\INFSCI1540-Project-Climate\climate> docker-compose up -d
[+] up 10/10
 ✓ Network climate_default              Created                                    0.1s
 ✓ Container climate-mysql-odb-server-1 Created                                    0.3s
 ✓ Container climate-web-server-1       Created                                    0.3s
 ✓ Container climate-mysql-dw-server-1  Created                                    0.3s
 ✓ Container zookeeper                  Created                                    0.3s
 ✓ Container climate-phpmyadmin-odb-1   Created                                    0.2s
 ✓ Container climate-phpmyadmin-dw-1    Created                                    0.2s
 ✓ Container broker                     Created                                    0.2s
 ✓ Container ksqldb-server              Created                                    0.1s
 ✓ Container ksqldb-cli                 Created                                    0.1s
```

3.2 Docker Compose File

```
#version: '2'

services:
  web-server:
    image: php:7.4.3-apache
    volumes:
      - "./html/:/var/www/html/"
    ports:
      - "8080:80"

  mysql-odb-server:
    image: mysql:8.0.23
    platform: linux/amd64
    environment:
      MYSQL_ROOT_PASSWORD: secret
    volumes:
      - mysql-odb-data:/var/lib/mysql
    ports:
      - "13306:3306"

  mysql-dw-server:
    image: mysql:8.0.23
    platform: linux/amd64
    environment:
      MYSQL_ROOT_PASSWORD: secret
    volumes:
      - mysql-dw-data:/var/lib/mysql
    ports:
      - "23306:3306"

  phpmyadmin-odb:
    image: phpmyadmin/phpmyadmin:5.0.1
    platform: linux/amd64
    environment:
      PMA_HOST: mysql-odb-server
      PMA_USER: root
      PMA_PASSWORD: secret
    ports:
      - "15000:80"
    depends_on:
      - mysql-odb-server

  phpmyadmin-dw:
```

```
    image: phpmyadmin/phpmyadmin:5.0.1
    platform: linux/amd64
    environment:
      PMA_HOST: mysql-dw-server
      PMA_USER: root
      PMA_PASSWORD: secret
    ports:
      - "25000:80"
    depends_on:
      - mysql-dw-server

  zookeeper:
    image: confluentinc/cp-zookeeper:7.9.4
    platform: linux/amd64
    hostname: zookeeper
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  broker:
    image: confluentinc/cp-enterprise-kafka:7.9.4
    platform: linux/amd64
    hostname: broker
    container_name: broker
    depends_on:
      - zookeeper
    ports:
      - "29092:29092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://broker:9092,PLAINTEXT_HOST://localhost
    :29092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1

  ksqldb-server:
    image: confluentinc/cp-ksqldb-server:7.9.6-1-ubi8
```

```
    platform: linux/amd64
    hostname: ksqldb-server
    container_name: ksqldb-server
    depends_on:
      - broker
    ports:
      - "8088:8088"
    environment:
      KSQL_LISTENERS: http://0.0.0.0:8088
      KSQL_BOOTSTRAP_SERVERS: broker:9092
      KSQL_KSQL_LOGGING_PROCESSING_STREAM_AUTO_CREATE: "true"
      KSQL_KSQL_LOGGING_PROCESSING_TOPIC_AUTO_CREATE: "true"
      CONFLUENT_SUPPORT_METRICS_ENABLE: "false"

  ksqldb-cli:
    image: confluentinc/cp-ksqldb-cli:7.9.6-1-ubi8
    platform: linux/amd64
    container_name: ksqldb-cli
    depends_on:
      - broker
      - ksqldb-server
    entrypoint: /bin/sh
    tty: true

volumes:
  mysql-odb-data:
  mysql-dw-data:
```

4. Data Preparation Process, Cleaning, and Transformation

The data preparation process is implemented as a multi-stage pipeline.

4.1 Step 1: CSV Ingestion into Kafka

The script `dly_producer.py` reads the source CSV file and publishes one JSON message per record to the Kafka topic `NORMAL_DLY_RAW`. During this step, the script performs basic type conversion. Numerical values such as elevation, latitude, longitude, and the three climate measures are converted from text into numeric Python values before the record is published.

4.2 Step 2: Loading Raw Operational Data into ODB

The script `consumer.py` consumes messages from `NORMAL_DLY_RAW` and writes them into the operational database table `climate_normal_daily_raw`. In this stage, the process performs the following cleaning operations:

- parsing the date field from `YYYYMMDD` into SQL `DATE` format,
- converting numeric values to decimal types,
- skipping malformed messages when required attributes are missing,
- using `ON DUPLICATE KEY UPDATE` to avoid duplicate station-date records.

The ODB serves as a structured staging area between raw stream ingestion and dimensional loading.

4.3 Step 3: ksqlDB Stream Cleaning and Transformation

The repository includes a ksqlDB layer that transforms the raw Kafka topic into analysis-ready streams. First, the raw JSON topic is registered as the ksql stream `normal_dly_raw`. Then the file `create_clean_stream.sql` derives additional time attributes from the raw date string and casts the climate fields into doubles.

```
CREATE STREAM normal_dly_clean AS
SELECT
  station_id,
  station_name,
  elevation,
  latitude,
  longitude,
  norm_date,
  CAST(SUBSTRING(norm_date, 1, 4) AS INTEGER) AS year_num,
  CAST(SUBSTRING(norm_date, 5, 2) AS INTEGER) AS month_num,
  SUBSTRING(norm_date, 1, 6) AS norm_month,
  CAST(dly_tmin_normal AS DOUBLE) AS dly_tmin_normal,
  CAST(dly_tmax_normal AS DOUBLE) AS dly_tmax_normal,
  CAST(mtd_prcp_normal AS DOUBLE) AS mtd_prcp_normal
FROM normal_dly_raw
EMIT CHANGES;
```

This transformation is important because it introduces month-level and year-level attributes that are later used for summary aggregation.

4.4 Step 4: Loading the Star Schema

The script `load.py` reads the cleaned operational data from ODB and populates the warehouse star schema. The process:

- extracts unique stations and upserts them into `dim_station`,
- extracts unique dates and inserts them into `dim_date`,

- maps station identifiers to surrogate keys,
- loads the fact table `fact_daily_climate_normal`.

This loading step also uses idempotent insertion logic so the process can be rerun without creating duplicate warehouse records.

5. STAR Schema Design

The data warehouse uses a classical star schema with one fact table and two dimension tables.

5.1 Schema Overview

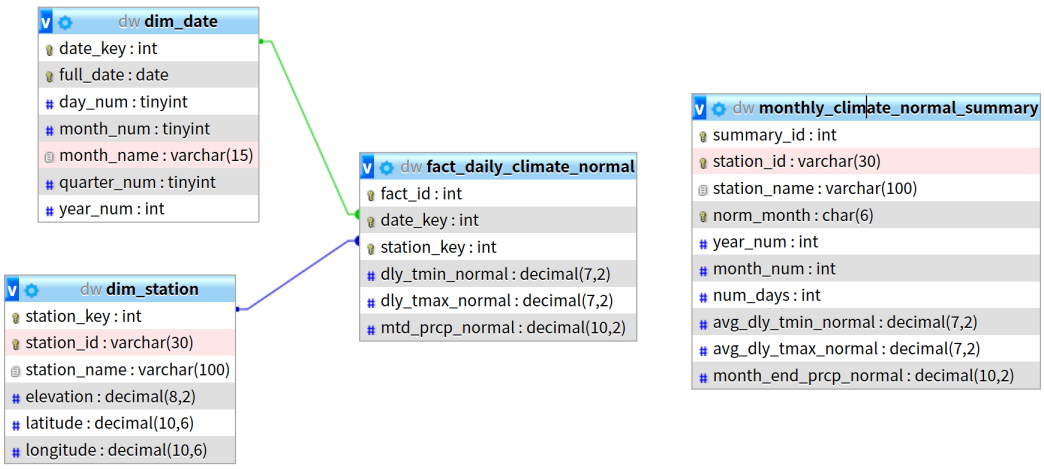


Figure 1: STAR schema design for the climate data warehouse.

The grain of the fact table is one climate normal record for one station on one date.

5.2 Dimension Table: `dim_date`

```
CREATE TABLE IF NOT EXISTS dim_date (
    date_key INT NOT NULL PRIMARY KEY,
    full_date DATE NOT NULL,
    day_num TINYINT NOT NULL,
    month_num TINYINT NOT NULL,
    month_name VARCHAR(15) NOT NULL,
    quarter_num TINYINT NOT NULL,
    year_num INT NOT NULL,
    CONSTRAINT uq_dim_date_full_date UNIQUE (full_date)
);
```

5.3 Dimension Table: `dim_station`

```sql
CREATE TABLE IF NOT EXISTS dim_station (
    station_key INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    station_id VARCHAR(30) NOT NULL,
    station_name VARCHAR(100) NOT NULL,
    elevation DECIMAL(8,2) NULL,
    latitude DECIMAL(10,6) NULL,
    longitude DECIMAL(10,6) NULL,
    CONSTRAINT uq_dim_station_station_id UNIQUE (station_id)
);
```

5.4 Fact Table: `fact_daily_climate_normal`

```sql
CREATE TABLE IF NOT EXISTS fact_daily_climate_normal (
    fact_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    date_key INT NOT NULL,
    station_key INT NOT NULL,
    dly_tmin_normal DECIMAL(7,2) NULL,
    dly_tmax_normal DECIMAL(7,2) NULL,
    mtd_prcp_normal DECIMAL(10,2) NULL,
    CONSTRAINT uq_fact_station_date UNIQUE (station_key, date_key),
    CONSTRAINT fk_fact_daily_normal_date
        FOREIGN KEY (date_key) REFERENCES dim_date(date_key),
    CONSTRAINT fk_fact_daily_normal_station
        FOREIGN KEY (station_key) REFERENCES dim_station(station_key)
);
```

5.5 SQL Statements Used to Populate the STAR Schema

The warehouse is populated through SQL statements executed by `load.py`. The main statements are listed below.

Populate `dim_station`

```sql
INSERT INTO dim_station (
    station_id, station_name, elevation, latitude, longitude
) VALUES (%s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    station_name = VALUES(station_name),
    elevation = VALUES(elevation),
    latitude = VALUES(latitude),
    longitude = VALUES(longitude);
```

Populate dim_date

```
INSERT IGNORE INTO dim_date (
    date_key, full_date, day_num, month_num, month_name, quarter_num, year_num
) VALUES (%s, %s, %s, %s, %s, %s, %s);
```

Populate fact_daily_climate_normal

```
INSERT INTO fact_daily_climate_normal (
    date_key, station_key, dly_tmin_normal, dly_tmax_normal, mtd_prcp_normal
) VALUES (%s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    dly_tmin_normal = VALUES(dly_tmin_normal),
    dly_tmax_normal = VALUES(dly_tmax_normal),
    mtd_prcp_normal = VALUES(mtd_prcp_normal);
```

6. Data Streams

Following the data warehousing workflow introduced in Lab 7 and the ksqlDB-based stream processing approach in Lab 9, our project uses Kafka and ksqlDB to support real-time ingestion and aggregation of climate data before loading the processed results into the final data warehouse.

6.1 Overview of the Stream Pipeline

The data stream pipeline in our project consists of four main stages:

1. Climate observation records are read from the prepared source file.
2. A Python producer sends these records to a Kafka topic.
3. ksqlDB defines the Kafka topic as a structured stream and performs continuous processing and aggregation.
4. The aggregated results are loaded into the MySQL data warehouse for multidimensional analysis and OLAP queries.

Compared with the original Lab 7 workflow, our project replaces the intermediate `odb.transaction` layer with a ksqlDB stream-processing layer. This allows the system to process incoming events directly from Kafka and generate aggregated results in real time.

6.2 Source Data Stream

The source data stream contains cleaned climate observation records. Each record represents one weather observation event and includes attributes such as observation date, station identifier, location, measurement type, and observed value.

These records are produced by a Python data producer and sent to a Kafka topic. In our project, this topic serves as the entry point of the streaming pipeline.

6.3 Kafka Topic and Stream Definition

After the producer sends the climate records to Kafka, ksqlDB is used to associate a schema with the underlying topic. In this way, raw Kafka messages are interpreted as structured stream records.

The stream definition logically represents the incoming climate events and acts as the streaming equivalent of the original transaction-level input layer. Instead of inserting incoming records into an ODB transaction table first, our project directly defines them as a ksqlDB stream.

6.4 Continuous Query Processing

To monitor incoming events, we use a continuous query over the climate event stream. This query continuously outputs new rows as data arrives in the Kafka topic.

This mechanism is useful for validating that the data producer is sending records correctly and that the stream pipeline is functioning as expected. It also demonstrates the real-time nature of the system, since newly produced climate observations immediately become visible in the stream query results.

6.5 Continuous Aggregation with ksqlDB

After the climate event stream is created, ksqlDB is used to define a continuous aggregation query. This query groups incoming records by selected attributes and incrementally computes aggregate values as new events arrive.

For our climate data warehouse, this aggregation step can be used to compute summarized measures such as monthly average temperature, monthly maximum temperature, monthly minimum temperature, and monthly total precipitation by station.

The aggregated output is maintained as a ksqlDB table. This table replaces the aggregation logic that was originally implemented in Lab 7 through the `odb_producer.py` script and the SQL query over the `transaction` table.

6.6 Loading Aggregated Results into the Data Warehouse

The aggregated results produced by ksqlDB are then loaded into the MySQL data warehouse. These results populate the fact table and support the creation of pre-aggregated summary tables.

The MySQL data warehouse remains the final storage layer of the project. It is used to maintain the STAR schema, support joins with dimension tables, and execute OLAP queries for climate trend analysis.

6.7 Benefits of the Modified Stream Design

This modified data stream design provides several advantages:

- It simplifies the original Lab 7 pipeline by removing the intermediate ODB transaction storage layer.

- It incorporates real-time stream processing using ksqlDB.
- It supports continuous aggregation directly on top of Kafka streams.
- It better demonstrates the integration of Kafka, ksqlDB, and MySQL in a modern data engineering workflow.

Overall, the modified stream architecture allows our project to combine the data warehousing logic of Lab 7 with the stream-processing capabilities of Lab 9, resulting in a more efficient and more scalable climate data pipeline.

7. Specification of Pre-Aggregated Summary Tables

The project includes a monthly pre-aggregated summary table in the warehouse and a corresponding aggregated table in ksqlDB.

7.1 Warehouse Summary Table DDL

```
CREATE TABLE IF NOT EXISTS monthly_climate_normal_summary (
    summary_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    station_id VARCHAR(30) NOT NULL,
    station_name VARCHAR(100) NOT NULL,
    norm_month CHAR(6) NOT NULL,
    year_num INT NOT NULL,
    month_num INT NOT NULL,
    num_days INT NULL,
    avg_dly_tmin_normal DECIMAL(7,2) NULL,
    avg_dly_tmax_normal DECIMAL(7,2) NULL,
    month_end_prcp_normal DECIMAL(10,2) NULL,
    CONSTRAINT uq_monthly_normal_station_month UNIQUE (station_id, norm_month)
);
```

7.2 SQL for Creation and Population of Summary Data in ksqlDB

```
CREATE TABLE normal_dly_monthly_summary
WITH (
  KEY_FORMAT = 'JSON',
  VALUE_FORMAT = 'JSON'
) AS
SELECT
  station_id,
  AS_VALUE(station_id) AS station_id_v,
  station_name,
  AS_VALUE(station_name) AS station_name_v,
```

```
  norm_month,
  AS_VALUE(norm_month) AS norm_month_v,
  year_num,
  AS_VALUE(year_num) AS year_num_v,
  month_num,
  AS_VALUE(month_num) AS month_num_v,
  COUNT(*) AS num_days,
  AVG(dly_tmin_normal) AS avg_dly_tmin_normal,
  AVG(dly_tmax_normal) AS avg_dly_tmax_normal,
  MAX(mtd_prcp_normal) AS month_end_prcp_normal
FROM normal_dly_clean
GROUP BY station_id, station_name, norm_month, year_num, month_num
EMIT CHANGES;
```

7.3 Loading Summary Data into the Warehouse

The script `load_monthly_summary.py` consumes the summary topic and upserts the aggregated rows into the warehouse summary table using the following SQL pattern.

```
INSERT INTO monthly_climate_normal_summary (
    station_id,
    station_name,
    norm_month,
    year_num,
    month_num,
    num_days,
    avg_dly_tmin_normal,
    avg_dly_tmax_normal,
    month_end_prcp_normal
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    station_name = VALUES(station_name),
    year_num = VALUES(year_num),
    month_num = VALUES(month_num),
    num_days = VALUES(num_days),
    avg_dly_tmin_normal = VALUES(avg_dly_tmin_normal),
    avg_dly_tmax_normal = VALUES(avg_dly_tmax_normal),
    month_end_prcp_normal = VALUES(month_end_prcp_normal);
```

8. OLAP Data Warehouse Queries

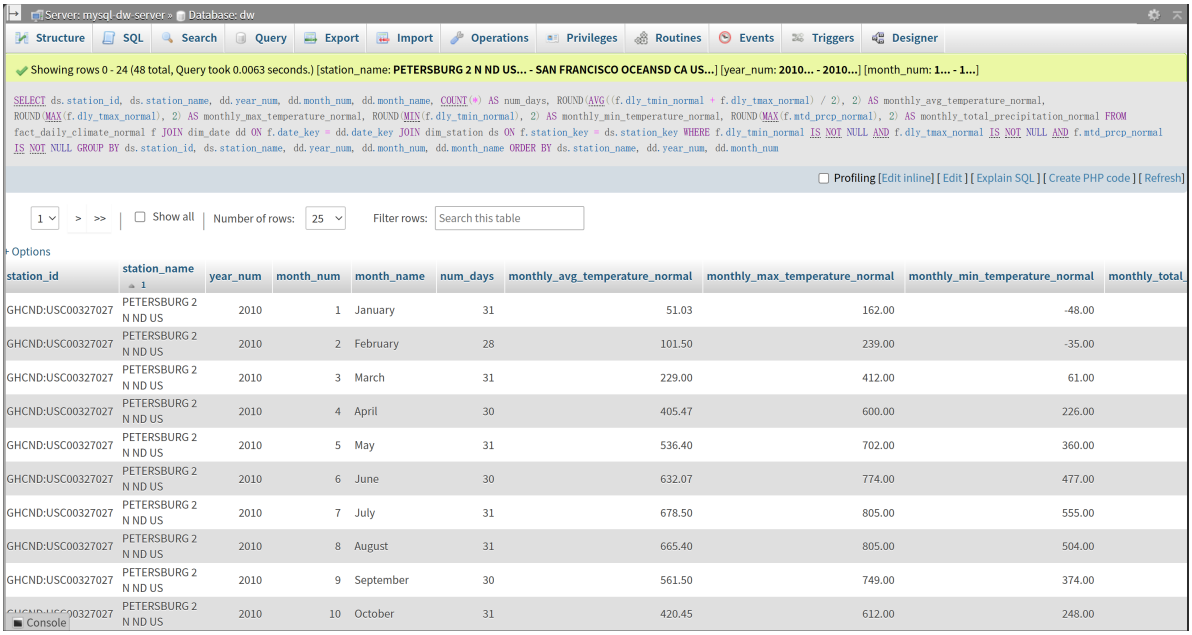
This section presents five OLAP queries executed against the climate data warehouse. Each query uses the warehouse fact table together with the date and station dimensions to support station comparison, temporal aggregation, and climate pattern analysis.

8.1 Query 1: Monthly Climate Summary by Station

```sql
USE dw;

-- OLAP Query 1:
-- Monthly average temperature, monthly max temperature,
-- monthly min temperature, and monthly total precipitation by station.
SELECT
    ds.station_id,
    ds.station_name,
    dd.year_num,
    dd.month_num,
    dd.month_name,
    COUNT(*) AS num_days,
    ROUND(AVG((f.dly_tmin_normal + f.dly_tmax_normal) / 2), 2)
        AS monthly_avg_temperature_normal,
    ROUND(MAX(f.dly_tmax_normal), 2)
        AS monthly_max_temperature_normal,
    ROUND(MIN(f.dly_tmin_normal), 2)
        AS monthly_min_temperature_normal,
    ROUND(MAX(f.mtd_prcp_normal), 2)
        AS monthly_total_precipitation_normal
FROM fact_daily_climate_normal f
JOIN dim_date dd
    ON f.date_key = dd.date_key
JOIN dim_station ds
    ON f.station_key = ds.station_key
WHERE
    f.dly_tmin_normal IS NOT NULL
    AND f.dly_tmax_normal IS NOT NULL
    AND f.mtd_prcp_normal IS NOT NULL
GROUP BY
    ds.station_id,
    ds.station_name,
    dd.year_num,
    dd.month_num,
    dd.month_name
ORDER BY
    ds.station_name,
    dd.year_num,
    dd.month_num;
```

Query 1 aggregates daily fact records into monthly station-level summaries. It calculates the number of daily observations, monthly average temperature, monthly maximum temperature, monthly minimum temperature, and monthly precipitation normal for each station. The result



Server: mysql-dw-server » Database: dw

Structure | SQL | Search | Query | Export | Import | Operations | Privileges | Routines | Events | Triggers | Designer

Showing rows 0 - 24 (48 total, Query took 0.0063 seconds.) [station_name: PETERSBURG 2 N ND US... - SAN FRANCISCO OCEANSD CA US...] [year_num: 2010... - 2010...] [month_num: 1... - 1...]

```
SELECT ds.station_id, ds.station_name, dd.year_num, dd.month_num, dd.month_name, COUNT(*) AS num_days, ROUND(AVG((f.dly_tmin_normal + f.dly_tmax_normal) / 2), 2) AS monthly_avg_temperature_normal,
ROUND(MAX(f.dly_tmax_normal), 2) AS monthly_max_temperature_normal, ROUND(MIN(f.dly_tmin_normal), 2) AS monthly_min_temperature_normal, ROUND(MAX(f.mtd_prcp_normal), 2) AS monthly_total_precipitation_normal FROM
fact_daily_climate_normal f JOIN dim_date dd ON f.date_key = dd.date_key JOIN dim_station ds ON f.station_key = ds.station_key WHERE f.dly_tmin_normal IS NOT NULL AND f.dly_tmax_normal IS NOT NULL AND f.mtd_prcp_normal
IS NOT NULL GROUP BY ds.station_id, ds.station_name, dd.year_num, dd.month_num, dd.month_name ORDER BY ds.station_name, dd.year_num, dd.month_num
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

1 > >> | Show all | Number of rows: 25 Filter rows: Search this table

+ Options

station_id	station_name	year_num	month_num	month_name	num_days	monthly_avg_temperature_normal	monthly_max_temperature_normal	monthly_min_temperature_normal	monthly_total_
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	1	January	31	51.03	162.00	-48.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	2	February	28	101.50	239.00	-35.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	3	March	31	229.00	412.00	61.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	4	April	30	405.47	600.00	226.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	5	May	31	536.40	702.00	360.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	6	June	30	632.07	774.00	477.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	7	July	31	678.50	805.00	555.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	8	August	31	665.40	805.00	504.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	9	September	30	561.50	749.00	374.00	
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	10	October	31	420.45	612.00	248.00	

Console

Figure 2: Result of Query 1: monthly climate summary by station.

is meaningful because it converts daily warehouse facts into a compact monthly view, making it easier to compare seasonal climate patterns across all stations.

8.2 Query 2: Coastal Seattle vs. Inland Petersburg Monthly Comparison

```
-- OLAP Query 2:
-- Compare similar-latitude coastal and inland stations:
-- Seattle Urban Site, WA vs Petersburg 2 N, ND.
-- This query pivots the two stations into the same monthly row so their
-- temperature and precipitation differences are easy to compare.
WITH monthly_station AS (
    SELECT
        ds.station_id,
        ds.station_name,
        CASE
            WHEN ds.station_id = 'GHCND:USW00024281' THEN 'Coastal Seattle'
            WHEN ds.station_id = 'GHCND:USC00327027' THEN 'Inland Petersburg'
        END AS comparison_group,
        dd.year_num,
        dd.month_num,
        dd.month_name,
        ROUND(AVG((f.dly_tmin_normal + f.dly_tmax_normal) / 2), 2)
            AS monthly_avg_temperature_normal,
        ROUND(MAX(f.mtd_prcp_normal), 2)
            AS monthly_total_precipitation_normal
```

```
    FROM fact_daily_climate_normal f
    JOIN dim_date dd
        ON f.date_key = dd.date_key
    JOIN dim_station ds
        ON f.station_key = ds.station_key
    WHERE ds.station_id IN ('GHCND:USW00024281', 'GHCND:USC00327027')
    GROUP BY
        ds.station_id,
        ds.station_name,
        comparison_group,
        dd.year_num,
        dd.month_num,
        dd.month_name
)
SELECT
    year_num,
    month_num,
    month_name,
    MAX(CASE WHEN comparison_group = 'Coastal Seattle'
        THEN monthly_avg_temperature_normal END) AS seattle_avg_temperature,
    MAX(CASE WHEN comparison_group = 'Inland Petersburg'
        THEN monthly_avg_temperature_normal END) AS petersburg_avg_temperature,
    ROUND(
        MAX(CASE WHEN comparison_group = 'Coastal Seattle'
            THEN monthly_avg_temperature_normal END)
        - MAX(CASE WHEN comparison_group = 'Inland Petersburg'
            THEN monthly_avg_temperature_normal END),
        2
    ) AS seattle_minus_petersburg_avg_temperature,
    MAX(CASE WHEN comparison_group = 'Coastal Seattle'
        THEN monthly_total_precipitation_normal END) AS seattle_total_precipitation,
    MAX(CASE WHEN comparison_group = 'Inland Petersburg'
        THEN monthly_total_precipitation_normal END) AS petersburg_total_precipitation,
    ROUND(
        MAX(CASE WHEN comparison_group = 'Coastal Seattle'
            THEN monthly_total_precipitation_normal END)
        - MAX(CASE WHEN comparison_group = 'Inland Petersburg'
            THEN monthly_total_precipitation_normal END),
        2
    ) AS seattle_minus_petersburg_precipitation
FROM monthly_station
GROUP BY
    year_num,
    month_num,
```

```
    month_name
ORDER BY
    year_num,
    month_num;
```

Showing rows 0 - 11 (12 total, Query took 0.0062 seconds.)

SELECT * FROM `query2`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code]

Show all | Number of rows: 25 Filter rows: Search this table

+ Options

year_num	month_num	month_name	seattle_avg_temperature	petersburg_avg_temperature	seattle_minus_petersburg_avg_temperature	seattle_total_precipitation	petersburg_total_precipitation	seattle_minus_petersburg_precipitation
2010	1	January	429.56	51.03	378.53	547.00	55.00	492.00
2010	2	February	442.57	101.50	341.07	352.00	44.00	308.00
2010	3	March	475.47	229.00	246.47	385.00	72.00	313.00
2010	4	April	514.05	405.47	108.58	279.00	98.00	181.00
2010	5	May	572.97	536.40	36.57	201.00	249.00	-48.00
2010	6	June	619.92	632.07	-12.15	157.00	376.00	-219.00
2010	7	July	661.52	678.50	-16.98	85.00	322.00	-237.00
2010	8	August	666.03	665.40	0.63	94.00	275.00	-181.00
2010	9	September	619.50	561.50	58.00	130.00	208.00	-78.00
2010	10	October	537.45	420.45	117.00	338.00	169.00	169.00
2010	11	November	464.98	250.98	214.00	598.00	81.00	517.00
2010	12	December	414.48	101.00	313.48	606.00	73.00	533.00

Figure 3: Result of Query 2: coastal Seattle versus inland Petersburg.

Query 2 compares Seattle and Petersburg by placing both stations in the same monthly rows. This pivoted layout makes the temperature and precipitation differences directly visible for each month. The result is useful because it highlights the contrast between a coastal station and an inland station, showing how geography can influence both normal temperature and precipitation.

8.3 Query 3: Seasonal Comparison Between San Francisco and Pittsburgh

```
-- OLAP Query 3:
-- Compare a west-coast major city station and an east/inland major city station:
-- San Francisco, CA vs Pittsburgh, PA.
-- This query rolls months into seasons to show broader regional differences.
WITH monthly_station AS (
    SELECT
        ds.station_id,
        ds.station_name,
        dd.year_num,
        dd.month_num,
        ROUND(AVG((f.dly_tmin_normal + f.dly_tmax_normal) / 2), 2)
            AS monthly_avg_temperature_normal,
        ROUND(MAX(f.dly_tmax_normal), 2)
            AS monthly_max_temperature_normal,
        ROUND(MIN(f.dly_tmin_normal), 2)
            AS monthly_min_temperature_normal,
        ROUND(MAX(f.mtd_prcp_normal), 2)
            AS monthly_total_precipitation_normal
    FROM fact_daily_climate_normal f
    JOIN dim_date dd
```

```sql
        ON f.date_key = dd.date_key
    JOIN dim_station ds
        ON f.station_key = ds.station_key
    WHERE ds.station_id IN ('GHCND:USC00047767', 'GHCND:USW00094823')
    GROUP BY
        ds.station_id,
        ds.station_name,
        dd.year_num,
        dd.month_num
),
seasonal_station AS (
    SELECT
        station_id,
        station_name,
        year_num,
        CASE
            WHEN month_num IN (12, 1, 2) THEN 'Winter'
            WHEN month_num IN (3, 4, 5) THEN 'Spring'
            WHEN month_num IN (6, 7, 8) THEN 'Summer'
            ELSE 'Fall'
        END AS season_name,
        CASE
            WHEN month_num IN (12, 1, 2) THEN 1
            WHEN month_num IN (3, 4, 5) THEN 2
            WHEN month_num IN (6, 7, 8) THEN 3
            ELSE 4
        END AS season_order,
        ROUND(AVG(monthly_avg_temperature_normal), 2)
            AS seasonal_avg_temperature,
        ROUND(MAX(monthly_max_temperature_normal), 2)
            AS seasonal_max_temperature,
        ROUND(MIN(monthly_min_temperature_normal), 2)
            AS seasonal_min_temperature,
        ROUND(SUM(monthly_total_precipitation_normal), 2)
            AS seasonal_total_precipitation
    FROM monthly_station
    GROUP BY
        station_id,
        station_name,
        year_num,
        season_name,
        season_order
)
SELECT
```

```
    year_num,
    season_name,
    station_name,
    seasonal_avg_temperature,
    seasonal_max_temperature,
    seasonal_min_temperature,
    ROUND(seasonal_max_temperature - seasonal_min_temperature, 2)
        AS seasonal_temperature_range,
    seasonal_total_precipitation
FROM seasonal_station
ORDER BY
    year_num,
    season_order,
    station_name;
```

Showing rows 0 - 7 (8 total, Query took 0.0047 seconds.)

SELECT * FROM `query3`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [

Show all | Number of rows: 25 Filter rows: Search this table

+ Options

	year_num	season_name	station_name	seasonal_avg_temperature	seasonal_max_temperature	seasonal_min_temperature	seasonal_temperature_range	seasonal_total_precipita
Edit Copy Delete	2010	Winter	PITTSBURGH INTL AP PA US	306.35	443.00	209.00	234.00	7
Edit Copy Delete	2010	Winter	SAN FRANCISCO OCEANSD CA US	514.67	591.00	447.00	144.00	12
Edit Copy Delete	2010	Spring	PITTSBURGH INTL AP PA US	501.98	749.00	255.00	494.00	10
Edit Copy Delete	2010	Spring	SAN FRANCISCO OCEANSD CA US	540.82	600.00	471.00	129.00	4
Edit Copy Delete	2010	Summer	PITTSBURGH INTL AP PA US	709.53	827.00	542.00	285.00	11
Edit Copy Delete	2010	Summer	SAN FRANCISCO OCEANSD CA US	578.16	635.00	516.00	119.00	
Edit Copy Delete	2010	Fall	PITTSBURGH INTL AP PA US	532.81	794.00	302.00	492.00	8
Edit Copy Delete	2010	Fall	SAN FRANCISCO OCEANSD CA US	577.36	647.00	466.00	181.00	3

Figure 4: Result of Query 3: seasonal comparison between San Francisco and Pittsburgh.

Query 3 rolls monthly climate values into seasonal summaries for San Francisco and Pittsburgh. It reports seasonal average temperature, maximum and minimum normal temperatures, seasonal temperature range, and total precipitation. The result is meaningful because it moves beyond monthly detail and shows broader regional differences, especially the contrast between San Francisco's milder coastal climate and Pittsburgh's stronger seasonal variation.

8.4 Query 4: Seattle vs. San Francisco Monthly Difference Ranking

```
-- OLAP Query 4:
-- Compare two west-coast stations at different latitudes:
-- Seattle, WA vs San Francisco, CA.
-- This query ranks months by the absolute temperature difference between them.
WITH monthly_station AS (
    SELECT
        ds.station_id,
        ds.station_name,
```

```
        ds.latitude,
        ds.longitude,
        dd.year_num,
        dd.month_num,
        dd.month_name,
        ROUND(AVG((f.dly_tmin_normal + f.dly_tmax_normal) / 2), 2)
            AS monthly_avg_temperature_normal,
        ROUND(MAX(f.mtd_prcp_normal), 2)
            AS monthly_total_precipitation_normal
    FROM fact_daily_climate_normal f
    JOIN dim_date dd
        ON f.date_key = dd.date_key
    JOIN dim_station ds
        ON f.station_key = ds.station_key
    WHERE ds.station_id IN ('GHCND:USW00024281', 'GHCND:USC00047767')
    GROUP BY
        ds.station_id,
        ds.station_name,
        ds.latitude,
        ds.longitude,
        dd.year_num,
        dd.month_num,
        dd.month_name
),
paired_months AS (
    SELECT
        sea.year_num,
        sea.month_num,
        sea.month_name,
        sea.latitude AS seattle_latitude,
        sf.latitude AS san_francisco_latitude,
        sea.monthly_avg_temperature_normal AS seattle_avg_temperature,
        sf.monthly_avg_temperature_normal AS san_francisco_avg_temperature,
        ROUND(sf.monthly_avg_temperature_normal
            - sea.monthly_avg_temperature_normal, 2)
            AS san_francisco_minus_seattle_temperature,
        sea.monthly_total_precipitation_normal AS seattle_total_precipitation,
        sf.monthly_total_precipitation_normal AS san_francisco_total_precipitation,
        ROUND(sf.monthly_total_precipitation_normal
            - sea.monthly_total_precipitation_normal, 2)
            AS san_francisco_minus_seattle_precipitation
    FROM monthly_station sea
    JOIN monthly_station sf
        ON sea.year_num = sf.year_num
```

```
        AND sea.month_num = sf.month_num
    WHERE sea.station_id = 'GHCND:USW00024281'
        AND sf.station_id = 'GHCND:USC00047767'
)
SELECT
    year_num,
    month_num,
    month_name,
    seattle_latitude,
    san_francisco_latitude,
    seattle_avg_temperature,
    san_francisco_avg_temperature,
    san_francisco_minus_seattle_temperature,
    seattle_total_precipitation,
    san_francisco_total_precipitation,
    san_francisco_minus_seattle_precipitation,
    RANK() OVER (
        ORDER BY ABS(san_francisco_minus_seattle_temperature) DESC
    ) AS temperature_difference_rank
FROM paired_months
ORDER BY
    temperature_difference_rank,
    year_num,
    month_num;
```

Showing rows 0 - 11 (12 total, Query took 0.0047 seconds.)

SELECT * FROM `query4`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

+ Options

year_num	month_num	month_name	seattle_latitude	san_francisco_latitude	seattle_avg_temperature	san_francisco_avg_temperature	san_francisco_minus_seattle_temperature	seattle_total_precipitation	san_francisco_total_precipitation	san_francisco_minus_seattle_precipitation
2010	12	December	47.650000	37.728000	414.48	510.45	95.97	606.00	397.00	-209.00
2010	11	November	47.650000	37.728000	464.98	552.02	87.04	598.00	255.00	-343.00
2010	7	July	47.650000	37.728000	661.52	577.50	-84.02	85.00	1.00	-84.00
2010	2	February	47.650000	37.728000	442.57	525.50	82.93	352.00	410.00	58.00
2010	1	January	47.650000	37.728000	429.56	508.05	78.49	547.00	418.00	-129.00
2010	8	August	47.650000	37.728000	666.03	590.50	-75.53	94.00	5.00	-89.00
2010	3	March	47.650000	37.728000	475.47	532.48	57.01	385.00	263.00	-122.00
2010	6	June	47.650000	37.728000	619.92	566.48	-53.44	157.00	21.00	-136.00
2010	10	October	47.650000	37.728000	537.45	587.00	49.55	338.00	106.00	-232.00
2010	9	September	47.650000	37.728000	619.50	593.05	-26.45	130.00	10.00	-120.00
2010	4	April	47.650000	37.728000	514.05	539.47	25.42	279.00	118.00	-161.00
2010	5	May	47.650000	37.728000	572.97	550.52	-22.45	201.00	64.00	-137.00

Figure 5: Result of Query 4: Seattle versus San Francisco monthly difference ranking.

Query 4 compares Seattle and San Francisco month by month and ranks the months by the absolute temperature difference between the two stations. It also includes the latitude of each station and the precipitation difference for the same month. The result is useful because it identifies the months when the climate gap between the two West Coast stations is strongest, helping connect station location with observed seasonal differences.

8.5 Query 5: Annual Climate Profile and Wettest Month by Station

```sql
-- OLAP Query 5:
-- Annual climate profile by station.
-- This query adds more OLAP variety by combining yearly averages,
-- annual temperature range, annual precipitation, and wettest month ranking.
WITH monthly_station AS (
    SELECT
        ds.station_id,
        ds.station_name,
        dd.year_num,
        dd.month_num,
        dd.month_name,
        ROUND(AVG((f.dly_tmin_normal + f.dly_tmax_normal) / 2), 2)
            AS monthly_avg_temperature_normal,
        ROUND(MAX(f.dly_tmax_normal), 2)
            AS monthly_max_temperature_normal,
        ROUND(MIN(f.dly_tmin_normal), 2)
            AS monthly_min_temperature_normal,
        ROUND(MAX(f.mtd_prcp_normal), 2)
            AS monthly_total_precipitation_normal
    FROM fact_daily_climate_normal f
    JOIN dim_date dd
        ON f.date_key = dd.date_key
    JOIN dim_station ds
        ON f.station_key = ds.station_key
    GROUP BY
        ds.station_id,
        ds.station_name,
        dd.year_num,
        dd.month_num,
        dd.month_name
),
annual_profile AS (
    SELECT
        station_id,
        station_name,
        year_num,
        ROUND(AVG(monthly_avg_temperature_normal), 2)
            AS annual_avg_temperature,
        ROUND(MAX(monthly_max_temperature_normal), 2)
            AS annual_max_temperature,
        ROUND(MIN(monthly_min_temperature_normal), 2)
            AS annual_min_temperature,
        ROUND(MAX(monthly_max_temperature_normal)
            - MIN(monthly_min_temperature_normal), 2)
```

```
            AS annual_temperature_range,
        ROUND(SUM(monthly_total_precipitation_normal), 2)
            AS annual_total_precipitation
    FROM monthly_station
    GROUP BY
        station_id,
        station_name,
        year_num
),
wettest_month AS (
    SELECT
        station_id,
        year_num,
        month_name AS wettest_month_name,
        monthly_total_precipitation_normal AS wettest_month_precipitation,
        ROW_NUMBER() OVER (
            PARTITION BY station_id, year_num
            ORDER BY monthly_total_precipitation_normal DESC, month_num
        ) AS precipitation_rank
    FROM monthly_station
)
SELECT
    ap.station_id,
    ap.station_name,
    ap.year_num,
    ap.annual_avg_temperature,
    ap.annual_max_temperature,
    ap.annual_min_temperature,
    ap.annual_temperature_range,
    ap.annual_total_precipitation,
    wm.wettest_month_name,
    wm.wettest_month_precipitation,
    RANK() OVER (
        PARTITION BY ap.year_num
        ORDER BY ap.annual_temperature_range DESC
    ) AS annual_temperature_range_rank
FROM annual_profile ap
JOIN wettest_month wm
    ON ap.station_id = wm.station_id
    AND ap.year_num = wm.year_num
WHERE wm.precipitation_rank = 1
ORDER BY
    ap.year_num,
    annual_temperature_range_rank,
```

```
    ap.station_name;
```

Showing rows 0 - 3 (4 total, Query took 0.0087 seconds.)

SELECT * FROM `query5`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Options

station_id	station_name	year_num	annual_avg_temperature	annual_max_temperature	annual_min_temperature	annual_temperature_range	annual_total_precipitation	wettest_month_name	wettest_month_precipitation	annual_temperature_range_rank
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	386.11	805.00	48.00	853.00	2022.00	June	376.00	1
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	512.67	827.00	209.00	618.00	3819.00	June	430.00	2
GHCND:USW00024281	SEATTLE URBAN SITE WA US	2010	534.88	772.00	355.00	417.00	3772.00	December	606.00	3
GHCND:USC00047767	SAN FRANCISCO OCEANSD CA US	2010	552.75	647.00	447.00	200.00	2068.00	January	418.00	4

Figure 6: Result of Query 5: annual climate profile and wettest month by station.

Query 5 builds an annual climate profile for each station from monthly summaries. It calculates annual average, maximum, and minimum temperature normals, annual temperature range, annual total precipitation, and the wettest month for each station. The result supports annual-level comparison by showing which stations have the largest temperature variability and how precipitation patterns differ across locations.